



INDIA.RUNS

Build what next India runs on

Team Name : Sigmoid

Team Leader Name : Rehaan Ahmad Khan

Problem Statement : Track 1 - Data & AI Challenge: Intelligent Candidate Discovery & Ranking

Solution Overview

An interpretable multi-stage scoring engine + grounded reasoning

A

Honeypot rejection

Impossible profiles (fabricated tenure, 0-month “expert” skills) are hard-rejected before ranking begins.

B

JD-fit score

Structured title + career-history substance, blended with dense semantic similarity capped at 30%.

C

Behavioral modifiers

Bounded multipliers for availability, notice period, location, open-source and job-hop stability.

D

Assembly

Enforce the validator's invariants, take the top 100, and attach plain-language reasoning.

Why this beats traditional matching

Traditional	Our approach
Keyword / BM25 overlap on skills	Title + career-history substance dominate
Pure embedding similarity	Embeddings blended at 30%, capped
Trusts every profile	Honeypots & stuffers hard-rejected
Skills only; ignores hireability	Behavioral signals modulate the score
Opaque score	Plain-language, fact-checked reasoning
GPU / API per candidate	CPU-only, no network, ~30s for 100k

JD Understanding & Candidate Evaluation

Every JD clause translated into an explicit, inspectable rule



Must-haves extracted from the JD

- Production embeddings retrieval — drift, index refresh, regressions
- Production vector DB / hybrid search (Pinecone, FAISS, Elastic...)
- Strong Python; ranking-eval frameworks (NDCG, MRR, MAP, A/B)
- Ideal: 6–8 yrs, 4–5 in applied ML at product (not services) companies
- Shipped a ranking / search / recsys system; Pune–Noida; active



“Do NOT want” — encoded as suppressors

- Keyword stuffers — AI skills under a non-AI title
- Title-chasers / job-hoppers (<1.5-yr stints)
- All-consulting careers (TCS / Infosys / Wipro / ...)
- CV / speech / robotics-only, without NLP / IR
- Pure research / no production; recent-LangChain-only AI
- Long notice period — the bar rises past 30 days



Signals that decide the order (most → least)

1

Current title class - strong AI / adjacent / other — the primary gate

2

Career-history substance - prose evidence of building retrieval / ranking in production — the tier-5 tell

3

Semantic similarity (dense) - recall for strong but keyword-light profiles

4

Availability / behavioral - recency, response rate, open-to-work (reorders 6 of the top 10)

5

Experience band & domain : 5–9 yrs · product-vs-consulting · NLP/IR-vs-CV

6

Hireability & validation - notice period, location / visa, GitHub / OSS

7

Recruiter interest - saves, views - mild, capped (popularity proxy)

Ranking Methodology

Retrieve → Score → Rank, driven by one auditable formula



1. Retrieve

Load all 100k; honeypot hard-reject (~44 impossible profiles) leaves the contention pool.



2. Score

One formula in pipeline.py - structured fit blended with cosine, then bounded modifiers.



3. Rank

Sort; clamp scores non-increasing by rank; tie-break by ascending candidate_id; take top 100.

The single scoring formula

```
final = blended_fit(structured, cosine)
      × stuffer_factor
      × availability_modifier (floor 0.5)
      × location · notice · external_validation
      × stability · desirability
```

```
blended_fit = 0.70*structured + 0.30*cosine
structured  = 0.35*title + 0.30*career + 0.15*YOE
             + 0.10*domain + 0.10*product
```

Combining the signals

Fit is additive: five sub-scores form one [0,1] structured score, blended with cosine.

Modifiers are multiplicative & bounded: they modulate, never dominate; a strong candidate keeps ≥50% credit even if quiet.

Structured weighted 0.70 so similarity can never promote a non-AI title - the core anti-trap choice.

Explainability & Data Validation

Decisions you can read, claims that are verified, traps that are caught



How decisions are explained

- Every candidate ships a 1–2 sentence, fact-grounded justification
- Cites specific facts: title, years, employer, named signal values
- Connects to specific JD requirements (production retrieval / ranking)
- Tone tracks the rank — lower ranks lead with honest concerns



Preventing hallucination

- Reasoning built only from fields that exist in the profile
- Template generator assembles from structured fields — nothing invented
- Verifier checks every named skill / employer / number, drops unverified
- Optional offline LLM upgrade passes through the same verifier



Suspicious / low-quality profiles

- ~80 honeypots designed; 44 flagged high-precision, hard-rejected (ship 0)
- 7,092 keyword stuffers penalized; best lands at 85,287 / 99,956
- Claim inflation (“expert” vs low assessment) is down-weighted
- Sentinel –1 / empty fields treated as neutral, never a penalty

End-to-End Workflow

A slow, untimed precompute feeds a fast, deterministic ranking step

OFFLINE · precompute (untimed, ~49 min)

build_jd_vector.py

build_embeddings.py (100k → vectors)

build_reasoning.py (top-150, verified)

artifacts/
embeddings · jd_vecs · reasoning

shipped in the repo ↓ loaded at run time



LIVE · rank.py (<5 min · CPU · no network)

1. load + honeypot reject

2. structured fit × cosine blend

3. stuffer + behavioral modifiers

4. combine → monotonic + tie-break

5. top 100 + grounded reasoning

6. assert 0 honeypots

submission.csv ✓

System Architecture

One source of truth for scoring, a reviewable rubric, validator-safe output

repo/

rank.py ← LIVE entry point (<5 min reproduce)
ranker/io_utils.py load .jsonl / .jsonl.gz (magic-byte sniff)
ranker/jd_spec.py JD rubric as explicit config
ranker/honeypot.py impossible-profile hard-reject
ranker/fit_score.py structured sub-scores + cosine blend
ranker/stuffer.py keyword-stuffer & claim-inflation penalty
ranker/signals.py availability / notice / location / OSS
ranker/dense.py precomputed embeddings → cosine to JD
ranker/pipeline.py SINGLE source of truth: final_score()
ranker/combine.py monotonic clamp + tie-break + CSV writer
ranker/reasoning.py grounded reasoning + verifier
precompute/ · artifacts/build scripts → embeddings, jd_vecs, cache

Design principles



One formula

pipeline.py is the only place final_score() lives — shipped ranking and cached reasoning can never drift apart.



Data-driven rubric

the JD requirements live in a reviewable config, not buried in code.



Validator invariants in code

monotonic score + tie-break enforced, not hoped for.



Graceful degradation

missing artifacts → structured-only ranking still runs.

Results & Performance

Validated by reading the output and by ablation — the metric is hidden

0

honeypots in top 100

0

keyword stuffers in top 100

100 / 100

show retrieval / ranking
evidence

~30 s

to rank 100k on a laptop CPU

Top-100 profile (faithful to the JD)

- 100 / 100 strong AI / ML / DS / NLP / Search titles
- 0 honeypots · 0 all-consulting careers · 0 keyword stuffers
- ~90 / 100 India (rest relocation-friendly); 74 in the 5–9 yr band
- Hireability lift: notice $\geq 90d$ 49 $\rightarrow$ 32; GitHub-validated 77 $\rightarrow$ 82
- Robustness (our proxy, no leaderboard): top-10 stable across blend weights; rank-1 invariant

Meeting the runtime & compute constraints

Constraint	Limit	Ours
Ranking runtime	≤ 5 min	~30 s
Memory	≤ 16 GB	$\ll (77 \text{ MB} + \text{JSONL})$
Compute	CPU only	CPU only
Network	off	off
Honeypots in top-100	$< 10\%$	0%

Technologies Used

Every tool justified by the constraint set: offline, CPU, ≤ 5 min, reproducible



Python 3.10

Required by the challenge; strong ecosystem — and the JD itself wants strong Python.



sentence-transformers · MiniLM-L6-v2

22M params, fast on CPU, fully offline; free at rank time because precomputed.



PyTorch (CPU build)

Backs the embedder — precompute only, never loaded during the ranking step.



NumPy

The only runtime dependency of rank.py — vectorized cosine similarity + sort.



Python stdlib (csv, json, gzip)

Keeps the live path dependency-light and trivially reproducible.

No vector DB / GPU / hosted LLM at rank time — the constraints forbid network & GPU, and a 100k pool doesn't need an external index. A 77 MB in-memory matrix is faster and simpler.

Submission Assets

Everything for Stage-3 reproduction is in the repo; the sandbox is the verification layer



GitHub repository

Code, shipped artifacts, README with a one-command reproduce.

```
github.com/RAK2315/redrob-hackathon
```



Reproduce command

≤5 min, CPU, no network. Deliverable: submission.csv (validated).

```
python rank.py --candidates ./candidates.jsonl --out ./submission.csv
```



Sandbox demo (required by spec)

Lightweight verification layer. Status: to add.

```
HF Space / Colab — ranker on a ≤100-candidate sample
```



Metadata + declarations

Team, compute flags, AI-tools declaration, methodology.

```
submission_metadata.yaml
```



INDIA.RUNS

Build what next India runs on

THANK YOU